MNN-Pocket-0.3B 深度解读与技术评估报告

> **对象**：淘宝 MNN 团队开源的 MNN-Pocket-0.3B（约 3 亿参数端侧稠密语言模型）
>
> **依据**：量子位公众号文章、`MNN-Pocket-0.3B-Technical-Report.pdf`（24 页，2026-10-26）、ModelScope 模型卡与模型仓库文件清单
>
> **定位**：技术团队评估 + 开发者上手指南
>
> **日期**：2026-11-09
>
> **来源标注约定**
>
> - `[文章]` 公众号原文
> - `[报告 §x]` 技术报告对应章节
> - `[模型卡]` ModelScope / Hugging Face 模型卡
> - `[推算]` 本报告基于公开配置与公式的计算，非原文数据
> - `[判断]` 本报告的分析意见

目录

1. 结论先行：一页纸速览
2. 背景：什么叫"手机应用原生大模型"
3. 模型规格与架构解剖
4. 训练流水线：4.1T 预训练 + 三段式后训练
5. 端侧推理工程：MNN 如何把 0.3B 跑快
6. 能力评测：数字背后的真实强弱项
7. 两条应用验证链路
8. 批判性评估：亮点、局限与待核实声明
9. 开发者上手指南
10. 行业定位与意义
11. 结论
12. 附录

---

1. 结论先行：一页纸速览

一句话结论：MNN-Pocket-0.3B 不是"又一个 0.3B 小模型"，而是一套以"手机应用资源预算"为第一性约束、把**模型结构—词表—数据配方—蒸馏—量化—推理引擎—草稿模型**打通的小模型工程范本。它在端侧效率上确实处于第一梯队，在"指令遵循 + 数学 + 代码"上明显越级；但**知识类任务是它的短板**，且部分传播数字（iPhone 18 Pro 350 tok/s、对比 llama.cpp、功耗）无法从技术报告中核实。

关键判断（按重要性排序）

#	判断	依据	置信度
1	核心创新是"协同设计"，而非单一模型结构	报告全篇；架构/词表/QAT/引擎/草稿模型联动	高
2	端侧效率第一梯队：173MB 部署包、228MB 峰值、M5 上 466 tok/s 解码	[报告 表3/表4]	高
3	能力长板是**指令遵循、中英数学、代码**，短板是**知识**（相对强弱；绝对分受基线口径影响）	[报告 表7] C-Eval 38.86 vs MiniCPM4 67.98	高
4	64K 词表是被低估的关键设计：在 300M 总预算下多换来约 9 层深度	[推算]	高
5	MOPD（多教师 On-Policy 蒸馏）贡献了 +14.2 的平均分，是后训练主引擎	[报告 §6.4] 35.79→50.00	高（增益真实；幅度可能含教师数据重叠，见 §8）
6	INT4 QAT 损失很小（-0.41），但分项有升有降、方向不稳定	[报告 表7]	中高
7	"32K 上下文"属于能力声明，**不是 300MB 预算下的可用上下文**：BF16 KV 在 32K 约 448 MiB	[推算]	高
8	所谓"8B 级效果"是**任务级目标**，不是通用能力对标；报告未设置 8B 基线	[文章] vs [报告]	高
9	iPhone 18 Pro 的 350 tok/s 来自演示视频，报告基准表中没有该设备	[文章/模型卡]	高
10	"比 llama.cpp 快 1.5–2.9×"未出现在技术报告中，无法交叉验证	[文章]	高
11	四个设计目标中的"功耗"在技术报告里没有独立评测章节，仅有文章给出的单机单负载数据	[报告] 全文核对	高
12	工程开放度很好：Apache-2.0、多阶段权重、GGUF、MNN 包、Dec 变体	[模型卡]	高

主观评分卡

维度	评分	理由
端侧工程与部署	A	真机数据完整、跨 4 种后端、含投机解码与负加速的诚实披露
模型—引擎协同设计	A	架构选型由真机吞吐/内存驱动，词表裁剪有依赖约束
训练方法	A-	数据分级 + SFT Loop + MOPD + Self-OPD/QAT，闭环完整；缺消融细节
评测严谨性	B	指标口径清楚，但基线复跑条件、去污染、统计显著性说明不足
功耗/热评估	C+	四目标之一，报告无章节；文章仅单设备、单负载、3 轮
开放度	A	Apache-2.0 + Base/SFT/Instruct/INT4/GGUF/Dec
适用广度	B-	仅中英；知识密集与长程推理明确不推荐

2. 背景：什么叫"手机应用原生大模型"

2.1 问题定义

文章把方向定义为**"手机应用原生大模型"**，关键词是"原生"：模型不是"能跑在手机上"，而是**与淘宝、闲鱼这类超级 App 的业务模块长期共处**——共享 CPU/GPU/NPU、内存带宽、电池和散热预算，并在持续调用、多任务并发下维持体验。

这带来四个可量化的设计目标：

维度	目标	工程含义
速度	解码 > 200 tok/s	对齐云端高速服务的生成体感
内存	推理峰值 < 300MB	模型模块总预算约 500MB；峰值压到 300MB 以下，为上下文增长与集成开销留余量
功耗	单次推理增量能耗 < 电池总量 1/10000；100 次 ≤ 1%	高频调用不显著影响续航
能力	指定业务场景达到 8B 级效果	任务级对齐，而非通用能力对标

[报告 §1] 另给出更贴近系统的表述：应用常驻内存需求约 200–300MB、推理峰值 < 300MB、生成速度接近 200 tok/s。注意这是两个口径："常驻"是应用侧总占用（RSS），"峰值"是加载模型并推理期间的 RSS 峰值（含权重）；本文后续的 228MB 即后者，不是增量。

2.2 为什么这个约束组合很苛刻

- **内存**：8–16GB 手机内存要由系统、相机、视频、电商页面共同分摊；模型峰值每多 100MB，业务侧就少一块图片/视频缓存。
- **带宽**：batch=1 解码是**权重流式读取**问题，不是算力问题。每 token 要把全部权重过一遍，模型越大，解码越受内存带宽限制。
- **功耗/热**：持续调用会导致降频，短时峰值吞吐不等于可持续吞吐。
- **能力**：0.3B 的容量天花板客观存在，必须靠数据质量、蒸馏和任务适配去"抠"。

[判断] "应用原生"本质上是一次**目标函数替换**：从"在充足资源下最大化 benchmark"换成"在严格资源与热约束下最大化业务可用性"。这个提法比单纯喊"端侧模型"更准确，也解释了后面所有设计取舍。

---

3. 模型规格与架构解剖

3.1 最终配置

项目	取值	来源
架构	Dense decoder-only，从 Qwen3 缩放而来（`Qwen3ForCausalLM`）	[报告 §2.2] [config.json]
层数	28	同上
隐藏维度	896	同上
注意力	GQA：14 个 Q 头 / 2 个 KV 头，head_dim 64	同上
QK-Norm	每层 q_norm / k_norm（dim 64），Qwen3 风格	[safetensors 头]
FFN	SwiGLU，中间维 2560	同上
词表	65,536（从 Qwen3 的约 151K 裁剪）	同上
词嵌入	输入/输出共享（`tie_word_embeddings=true`）；BF16 文件中 lm_head 与 embed_tokens 各存一份	同上
位置编码	RoPE，`rope_theta=1e6`，`max_position_embeddings=40960`（预留上限；训练/部署目标 32K；是否外推未说明）	[config.json]
部署上下文	32K	[报告/模型卡]
原生精度	BF16；部署为 W4A16（group size 64，非对称）	[报告 §4.2.3]
许可证	Apache-2.0	[模型卡]

3.2 参数量拆解（推算）

组成	计算	参数量	占比
词嵌入 / LM Head（共享）	65,536 × 896	58.72M	19.4%
每层注意力（Q/K/V/O）	0.803 + 0.115 + 0.115 + 0.803	1.835M	—
每层 FFN（gate/up/down）	3 × 896 × 2560	6.881M	—
单层合计	—	8.716M	2.9%
28 层合计	—	244.06M	80.6%
总计	—	**≈302.8M**	100%

> `[推算]` 与官方"约 300M"一致；差异来自 norm 等少量参数未计入。
>
> **与 723MB BF16 文件的对账**：`model.safetensors` 头显示 `lm_head.weight` 与 `model.embed_tokens.weight` 各为 [65536, 896]，即存储侧重复了一份 58.72M 的嵌入矩阵，**存储参数 361.55M × 2B = 723.1MB**，与文件大小完全吻合；但 `tie_word_embeddings=true`，**有效唯一参数为 302.8M**。这解释了"302.8M 参数却存 723MB"的表面矛盾。此外每层还有 q_norm/k_norm（各 64 维）与两个 RMSNorm，合计约 0.05M，可忽略。

3.3 被低估的关键设计：64K 词表

词表从 151,936 裁到 65,536（-56.9%），报告说这是为了降低嵌入/输出层参数与输出投影开销。`[报告 §2.3]` 其真实杠杆可以量化：

- 若保留 151K 词表、总参数仍锁定在约 300M，则留给 Transformer 层的预算只剩约 **164M**；
- 按每层 8.716M 计算，只够做 **约 19 层**；
- 64K 词表让层数可以做到 **28 层**——深度从 19 提到 28，正是报告里能力提升最明显的那一段（16→28 层显著降 loss）。

换言之，**词表裁剪不是"省点内存"，而是用 86.4K 个低频词条（151,936 − 65,536）换来了约 9 层深度**。这是小模型时代非常典型、但容易被忽略的"参数预算再分配"。

裁剪方法也值得一提：不是重训 tokenizer，而是

1. 用 6GiB 中英/代码/数学/指令语料统计词频；
2. 按词表预算贪心选取高频词条；
3. **保留全部特殊 token、256 个 byte-fallback token，以及每个入选 token 的完整 BPE merge 依赖链**；
4. 重映射到连续 ID。

`[判断]` "依赖感知裁剪"避免了"裁掉中间 merge 导致分词退化"的常见坑，也避免了从零训练 tokenizer 带来的初始化不匹配。这是工程上正确且低成本的做法。

3.4 为什么不是 MoE

报告明确选择稠密模型：MoE 虽然能用更少激活参数换容量，但**总参数更大、需要路由、专家访问不连续**，对 batch=1 的端侧解码不友好——端侧瓶颈是容量、带宽和访问局部性，而不是峰值算力。`[报告 §2.2]`

`[判断]` 在 0.3B 这个量级，MoE 的总参数开销和调度复杂度基本吃掉了收益，稠密是合理选择。

3.5 架构搜索：28 层是怎么定下来的

两阶段：

1. **定规模**：在 M5 iPad + MNN Metal 上扫 50M–1.3B，约束为峰值 <300MB、解码 >200 tok/s（W4A16、256 token 输入输出）。约 300M 是同时满足两条约束的最大候选（281.0MB / 358.3 tok/s）。
2. **定内部结构**：在 300M 预算内联合调深度/宽度/FFN/注意力。16→28 层显著降 loss、提能力；28→36 层能力不再提升，解码从 355 降到 323 tok/s，峰值内存还略升。最终选 28 层。

> **口径提醒（易误读）**：图 2(a) 的 ~300M 候选是 281MB / 358.3 tok/s，图 2(b) 的 28 层是 228MB / 355 tok/s，最终部署版是 228MB / 466 tok/s。三者分别是"规模筛选、结构对比、最终基准"，测试配置与 MNN 版本不同，不能横向比较；文章也明确说明架构选型的吞吐与最终部署配置不同。内存 228MB 在 (b) 与最终版一致，可放心引用。

[判断] 这是典型的"硬件在环"架构搜索：用真机吞吐和内存做约束，而不是只看 loss。对端侧团队来说，这个方法论比具体超参更有复用价值。

3.6 32K 上下文的真实内存代价（重要）

报告和文章都强调"支持 32K"，但**300MB 预算只在 512 token 上下文下测量**。KV Cache 随上下文线性增长，按公开配置估算：

```
每 token KV = 2(K/V) × 2(KV头) × 64(head_dim) × 28(层) × 2字节(BF16)
            = 14,336 B ≈ 14 KiB/token
```

上下文	BF16 KV Cache（MiB）	说明
512	7.0 MiB	报告测量点
4,096	56.0 MiB	估算总内存约 280MB，仍在 300MB 内
8,192	112.0 MiB	已超 300MB
32,768	**448.0 MiB（约 470MB）**	约为模型权重（173MB）的 2.7 倍

以 228MB（512 ctx）为基线，非 KV 部分约 221MB。要到 300MB，**总 KV 预算约 79MB（约 75MiB）**；扣除 512 token 已用的 7MB，**还能再增加约 5K token，总上下文约 5.5K**（未计运行时临时内存、分词器与框架开销）。即使把 KV 量化到 INT8，32K 仍需约 224MiB，依然放不进 300MB 预算。

> **预算口径**：文章同时提到"模型模块预算约 500MB"和"推理峰值 <300MB"。前者是模块可占用的总盘子，后者是团队设定的推理峰值目标，二者不矛盾。若按 500MB 总盘子算，8K（约 333MB）仍可容纳；本文按更严格的 300MB 峰值目标判定。

结论：[判断] "32K"是模型能力与训练目标，不是"手机端 300MB 峰值预算下可用的上下文长度"。实际可用区间大致是：**2K 左右较稳妥，4K 已接近上限（约 280MB），总上下文约 5.5K 是仅计 KV 的理论上限**。要用更长上下文，必须显式评估 KV 量化、内存水位与降频风险。这一点在文章和报告里都没有被强调，是本报告认为最需要补充的工程信息。

---

4. 训练流水线：4.1T 预训练 + 三段式后训练

整体流水线（[报告 图3]）：

```
Stage1 3T(4K) → Stage2 800B(4K) → Short Decay 200B(4K) → Long Decay 100B(32K)
   → Base → SFT(3.16M 样本) → SFT 模型
```

```
→ 四领域 RL 教师 + MOPD → 指令模型(BF16)
→ Self-OPD + QAT → INT4 部署模型
→ 同一指令模型初始化 → Dec 决策模型
```

4.1 预训练：数据工程驱动，而非堆 token

核心原则

- **Quality over Scale**：小模型对噪声、冗余 token 更敏感；
- **可控知识密度与难度**：不仅"干净"，还要"有信息量且学得动"；
- **分阶段领域平衡**：领域在容量受限模型里竞争更激烈，配比要随阶段调整。

五维数据标注

每篇文档标注：**领域（Domain）、质量（Quality）、知识密度（Density）、难度（Difficulty）、文本形式（Style）**。质量/密度/难度各分三级（Q1–Q3、D1–D3、H1–H3），形式分 Plain / Q&A / List / Table。

与传统"一个质量分 + 阈值过滤"不同，这里把五维组合成**联合桶**，作为阶段配比的基本调度单元。[判断] 这是把数据从"过滤后的语料"变成"可调度资产"的关键一步，也是小模型数据工程与大数据集时代的最大区别。

前三阶段配比（Long Decay 未公开）

维度	Stage 1（3T）	Stage 2（800B）	Short Decay（200B）
英文	66.4%	46.8%	39.0%
中文	17.4%	24.2%	22.7%
代码	8.2%	15.2%	19.9%
数学	8.0%	13.8%	18.4%
Plain	71.9%	70.5%	71.7%
Q&A	15.2%	23.2%	22.0%
List	10.2%	5.1%	4.9%
Table	2.7%	1.1%	1.5%
密度 D3/D2/D1	50.1/40.0/9.9	52.4/41.5/6.1	50.4/44.1/5.5
难度 H3/H2/H1	18.0/53.4/28.5	15.5/55.2/29.4	17.5/51.0/31.5
质量 Q3/Q2/Q1	48.1/45.8/6.1	51.2/45.5/3.3	50.4/46.3/3.3

> 密度/难度/质量行的三个数字依次为高/中/低（D3/D2/D1、H3/H2/H1、Q3/Q2/Q1）。[报告 图4] 只给出前三阶段的五维构成；第四阶段 Long Decay（100B、32K）的配比未披露，因此无法核对长上下文阶段的数据分布。

[判断] 三个信号值得注意：

1. **英文占比始终最高**（66%→39%），中文只有 17–24%，但模型中文能力并不弱，说明数据质量与后训练补偿起了作用；
2. 后两阶段**主动提高代码/数学占比**（8%→20%，8%→18%），对应"先建语言底座、再补硬能力"的顺序；
3. 质量 Q1 从 6.1% 压到 3.3%，说明退火阶段在"提纯"。

配方搜索方法

不是逐维调参，而是把配方定义为联合桶上的采样分布 `p_t(b)`，与学习率、token 预算、阶段边界**联合搜索**：

1. 用短程 proxy run 得到各候选配方的参数更新量 `Δθ_i`；
2. 用参数插值 `θ̃(α)=θ⁰+Σα_iΔθ_i` 粗筛大空间；

3. 对最强候选**用真实混合数据训练**精调（插值只用于粗筛）；
4. 按收敛、验证 loss、下游能力、跨域回归综合评估。

［判断］"插值粗筛 + 真训精调"是务实做法：纯插值会偏离真实训练轨迹，纯真训又太贵。这个方法对小团队复现配方很有参考价值。

训练超参

超参	Stage 1	Stage 2	Short Decay	Long Decay
上下文	4,096	4,096	4,096	32,768
Token 量	~3T	~800B	~200B	~100B
全局 batch	1024	1024	1024	128
每步 token	4.2M	4.2M	4.2M	4.2M
峰值 LR	3e-4	1e-4	4e-5	4e-5
LR 调度	Constant	Constant	Cosine	Cosine
Warmup	3,000	3,000	1,000	1,000
卡数	32	32	32	32
墙钟	7–8 周	~2 周	2–3 天	2–3 天

优化器 AdamW（β=0.9/0.95，ε=1e-15），weight decay 0.1，梯度裁剪 1.0。

成本侧写 ［推算］：

- 总 token 4.1T，TPP ≈ 4.1e12 / 3.03e8 ≈ **13,500 token/参数**；
- 按 6ND 粗估训练算力 ≈ 6 × 3.03e8 × 4.1e12 ≈ **7.4e21 FLOPs**；
- 四阶段墙钟合计约 10 周：32 卡 × 10 周 ≈ 1.94e8 GPU·秒，折合每卡约 **38 TFLOPS** 有效算力（仅 Stage 1 的 3T/7.5 周也约 38 TFLOPS，量级一致）。

［判断］这个量级说明两件事：一是**绝对训练成本很低**（几十张卡、两三个月），中等规模团队可承受；二是小模型 MFU 确实低，瓶颈在数据管线、评测与稳定性，而不是算力。报告也明确提到小模型训练 MFU 偏低。这印证了"端侧模型竞争在数据与工程，不在堆卡"。

4.2 SFT：3.16M 样本 + SFT Loop Engine

数据盘子

- 约 **3.16M 样本**、**1.83B assistant token**；
- 其中 **116.6K** 条来自 SFT Loop Engine 的定向合成；
- 三层分类体系：**1,511 个细类 / 154 个中类 / 6 个 family**。

Family	样本	Token	Token 占比
多语言创作与开放生成	607K	289M	15.81%
数学与定量推理	722K	393M	21.54%
编程与算法	801K	433M	23.70%
应用开发与通用 QA	247K	148M	8.08%
通用中文任务	450K	337M	18.43%
STEM QA 与科学计算	334K	227M	12.45%

Capability	样本	Token	Token 占比
Code	970K	558M	30.58%
Knowledge QA	747K	482M	26.41%
Mathematics	799K	443M	24.26%
Creative Writing	378K	250M	13.67%
Instruction Following	**90.0K**	**23.6M**	**1.29%**
Others	178K	69.2M	3.79%

> 占比按 assistant token 归一（报告明确说明），不是样本占比。例如多语言创作 607K 样本占样本数 19.2%，但 token 占比 15.81%。

[判断] 知识 QA 占 26.41% 的 SFT token，但最终 C-Eval/CMMLU 仍是短板。这不矛盾：SFT 数据量解决"会不会按格式答"，不解决"参数里有没有知识"；0.3B 的知识容量主要取决于预训练，且 C-Eval 覆盖学科极广，少量 QA 数据无法补齐。这也说明 MOPD 为什么把 Knowledge 单列一个教师。

[判断] 一个反直觉点：**指令遵循数据只占 1.29% 的 token，但最终 IFEval/IFEval-zh 是全场最高**。这说明 IF 能力主要不是靠 SFT 数据堆出来的，而是靠后续的 IF 教师 + MOPD 注入。对做小模型后训练的团队，这是个有价值的信号：IF 数据"少而精 + 蒸馏"可能比"堆量"更有效。

SFT Loop Engine

结构是一个主 Agent + 两个 Skill + 三个 Subagent：

```
训练/评测 Skill → SFT checkpoint → 评测 → 能力评估
      ↑                                        ↓
  精炼数据 ← Subagent C(合成/去重/泄漏检查) ← Subagent B(归因聚合/配额) ← Subagent A(失败归类)
```

- Subagent A：把失败回答及其生成轨迹映射到 1,511 个细类；
- Subagent B：结合历史数据分布，分配各类别/语言的合成配额；
- Subagent C：**在看不到原始评测题的前提下**合成新样本，并做去重与泄漏筛查；
- 主 Agent 只维护流程状态与模型版本决策。

[判断] 这是把"评测驱动迭代"自动化的一次比较完整的工程实现。尤其"C 不接触原始评测题"这条约束，是防止自蒸馏式刷榜的关键设计。但注意：它只约束了**SFT 合成环节**，并没有覆盖预训练数据与教师 RL 数据对评测集的去污染（见 §8）。

4.3 MOPD：把四个专家压进一个学生

SFT 之后的问题是：在固定 0.3B 容量下，同时强化代码、数学、知识、指令遵循。联合多域 RL 会互相拉扯，顺序训练会依赖顺序。团队的做法是**多教师 On-Policy 蒸馏（MOPD）**：

1. 从同一 base 出发，用领域 RL 训出四个教师：**Code / Math / Knowledge / IF**；
2. 四个教师共享架构、tokenizer、词表、chat template（降低分布错配）；
3. 每条样本按来源领域标签 `d` 确定性路由到一个冻结教师；
4. 学生**自己生成 rollout**，教师在学生的前缀上提供 **Top-K token 分布**监督；
5. 损失用 **reverse KL**（而非 forward KL），并在教师 Top-K 集合上重归一化：

```
ℓ_RKL = Σ_{v∈V_K} p̄_θ(v) log( p̄_θ(v) / q̄_T(v) )
```

为什么是 reverse KL：forward KL 是 mode-covering，会逼学生覆盖教师的所有尾部模式；reverse KL 是 **mode-seeking**，更强调教师的主峰，适合容量受限的学生"选择性继承"。

效果：11 项平均 **35.79 → 50.00**（+14.21），10 项提升、1 项持平（MBPP 37.20）。最大增益：

任务	增益
MGSM-zh（中文数学）	**+28.00**
IFEval-zh（中文指令遵循）	**+25.17**
MATH500	**+22.40**

[判断] MOPD 是这篇工作后训练的真正主引擎。它的聪明之处在于：**训练时用四个专家，部署时只留一个学生**，不增加端侧存储与推理开销。这比"在端侧跑 MoE/多模型路由"务实得多。

但 +22~+28 的增益也异常大，需要结合 §8 的去污染问题一起看：教师 RL 的领域数据与这些 benchmark 是否重叠，报告没有说明。

4.4 QAT + Self-OPD：把 BF16 能力搬到 INT4

- 冻结 MOPD 后的 BF16 模型作为 **self-teacher**；
- 用 PTQ 初始化一个 W4A16 学生（非对称 INT4，group 64，16-bit 激活，embedding 与 LM head 共享量化权重）；
- 在**学生自己生成的上下文**上，用同样的 Top-K reverse KL 对齐 teacher/student；
- 只保留学生用于部署。

结果：INT4 平均 **49.59**，BF16 **50.00**，仅降 **0.41**。

[判断] 0.41 分的平均差距很小、方向不稳定；报告未给置信区间，无法判定是否统计显著（见 §6.5）。更有说服力的是分项：INT4 在 C-Eval（+1.41）、IFEval（+0.92）、MBPP（+1.00）上反而更高，说明**差异主要是评测噪声，而非系统性量化损失**。这对部署是好消息。

4.5 训练流水线总评

环节	做法	评价
预训练	4.1T / 4 阶段 / 五维数据桶 / 配方搜索	方法论完整，成本极低，值得复用
SFT	3.16M 样本 + Agent 闭环	自动化程度高，IF 数据占比低但有效
MOPD	4 教师 + 路由 + reverse KL	本报告认为是最有含金量的部分
QAT	Self-OPD + W4A16	损失可控，工程闭环
缺失	数据配比/蒸馏/量化消融、去污染说明、功耗评测（深度消融已有）	影响可信度，见 §8

---

5. 端侧推理工程：MNN 如何把 0.3B 跑快

5.1 Prefill 与 Decode 是两种问题

[报告 §5.1] 说得很清楚：

- **Prefill**：并行处理 prompt，以矩阵乘-矩阵乘和注意力为主，**吃算力**；
- **Decode（batch=1）**：逐 token 流式读取全部权重，**吃内存带宽与 kernel 调度开销**。

因此优化手段必须分阶段。

5.2 MNN 的具体优化

层面	手段
CPU	针对 Arm 点积/矩阵指令定制低比特内核
GPU	按架构优化缓存访问、计算分块、线程配置
NPU	高通 Hexagon 专用后端
图优化	QKV / FFN 算子融合、GPU 命令复用
内存	分阶段执行路径、KV Cache 后端特化、低内存模式
生成加速	投机解码（DFlash2，8-draft 专用 GPU kernel）
集成	统一模型转换与推理接口，跨芯片/OS

[判断] 这些不是"黑科技"，但胜在**分层完整**：从指令级内核到图级融合再到运行时调度，且针对 batch=1 的真实瓶颈（带宽/调度）而非训练视角（算力）。这正是"模型—引擎协同"的实际含义。

5.3 真机数据

测试条件：MNN 3.6.2、4 CPU 线程、低精度/低内存模式、INT4、greedy、预热 1 次测 5 次取中位数、prefill/decode 从空 KV 独立测。[报告 §5.2]

部署体积与峰值内存（iPad Pro M5，512 token）

模型	MNN 文件 (MB)	峰值内存 (MB)
MNN-Pocket-0.3B	**173**	**228**
Gemma-3-270M	162	272
ERNIE-4.5-0.3B	207	287
LFM2.5-350M	202	258
SmolLM2-360M	206	257
MiniCPM4-0.5B	247	305
Hunyuan-0.5B	307	449
Qwen3-0.6B	340	427

[推算] 仓库实际文件：`llm.mnn.weight` 163.54 MiB + `llm.mnn` 0.91 MiB + `tokenizer.mtok` 1.63 MiB + 配置 ≈ 166.1 MiB ≈ **174.2 MB（十进制）**，与报告"173MB"基本一致（差 0.7%），属于 MiB/MB 换算差异。

[判断] 注意 Gemma-3-270M 文件更小（162MB）但峰值内存更高（272MB），说明**部署体积 ≠ 运行内存**：词表、KV 布局、加载缓冲、算子临时内存都会影响。MNN-Pocket 的 228MB 是这一组里最低的，这才是"应用原生"更看重的指标。

> **注意**：所有单模型峰值内存数据都来自 iPad Pro M5；Android 只报告了整条语音链路的 1,704MB（§7.2），没有单独给出 LLM 在 Android 上的峰值内存。做 Android 预算时应自行实测，不要直接套用 228MB。

吞吐（512 token，tok/s）

设备	后端	阶段	MNN-Pocket	Qwen3-0.6B	MiniCPM4-0.5B	Gemma-3-270M
iPad Pro M5	Metal	Prefill	**8,721**	6,027	6,973	9,519
iPad Pro M5	Metal	Decode	**466**	286	357	455
iPad Pro M5	CPU	Prefill	2,228	1,396	1,818	3,793
iPad Pro M5	CPU	Decode	231	135	197	290
骁龙 8 Gen 5	OpenCL	Prefill	3,403	1,627	2,368	4,209
骁龙 8 Gen 5	OpenCL	Decode	**138**	93	117	118
骁龙 8 Gen 5	CPU	Prefill	930	522	682	1,953
骁龙 8 Gen 5	CPU	Decode	**251**	126	189	293

完整 8 模型表见［报告 表4］。

三个反直觉点：

1. **Android 上 CPU 解码（251）远快于 OpenCL（138）**。batch=1 解码是带宽/调度问题，4 线程 CPU 加上低比特内核可能比通用 GPU 后端更高效。［判断］这提醒开发者：**不要默认 GPU 更快，必须按设备/后端实测选路。**
2. **Gemma-3-270M 在多数吞吐项领先**（M5 prefill 9,519、Android CPU decode 293 等 8 项中的 6 项），说明吞吐与任务能力不是同一维度。它的能力分数见 §6（本组最低，且多个 0.00 提示基线口径可能有问题），选型仍应以任务评测为准。
3. **MNN-Pocket 的 prefill 不是最快，但 decode 很强**：M5 Metal decode 466 tok/s，在 8 个模型里排第三（LFM2.5-350M 504、MNN 466、Gemma-3-270M 455），比同量级的 Qwen3-0.6B（286）高 63%。对交互式生成，decode 更决定体感。

> **目标达成情况**：团队定义的解码目标是 >200 tok/s。iPad Metal（466）、iPad CPU（231）、Android CPU（251）达标；**Android OpenCL 只有 138，未达标**。报告没有解释这条路径为何不达标，但从数据看，Android 上应优先选 CPU 后端做解码。
>
> 关于文章"在骁龙 8 Gen 5 CPU 后端 decode 251 tok/s"——与报告一致。但文章把 OpenCL 的 138 略过不谈，读者容易误以为所有后端都很快。

5.4 投机解码：有收益，但不是万能

- 方案：**DFlash2**，一次并行提出一个 block（B=8），目标模型一次前向验证；
- 草稿模型共享目标的 token embedding 与 LM head，用 D-PACE 的位置加权损失训练；
- 训练基于 SpecForge，prompt 来自 Open-PerfectBlend + 500K 后训练样本；
- 两种草稿：MNN-trained（BF16 目标）与 MNN-INT4-trained（INT4 目标）。

接受长度（AL）与加速：

设备	工作负载	AR tok/s	平均 AL	投机 tok/s	加速
iPad Pro M5	Math	435	4.49	709	**1.63×**
iPad Pro M5	Code	—	5.73	895	**2.06×**
iPad Pro M5	Chat	—	3.51	547	1.26×
iPad Pro M5	平均	—	4.57	717	**1.65×**
骁龙 8 Gen 5	Math	135	4.48	159	1.17×
骁龙 8 Gen 5	Code	—	5.72	186	1.38×
骁龙 8 Gen 5	Chat	—	3.49	119	**0.88×（负加速）**
骁龙 8 Gen 5	平均	—	4.56	155	1.15×

> **口径缺失**：报告未标注该表的测试后端、上下文长度与目标精度。Android 的 AR 135 tok/s 与表 4 的 OpenCL 138 接近，推测走的是 OpenCL；CPU 路径（251）未测。因此"Android 聊天负加速"至少应限定为"OpenCL 路径下"。

[判断] 这组数据很诚实：**代码/数学收益大，聊天收益小，Android OpenCL 上聊天甚至变慢**。原因是 AL 相近时，草稿/验证的固定开销在低带宽设备上占比更高；CPU 路径未测，不能外推。开发者应把投机解码当作**按负载开关**的能力，而不是默认开启。

5.5 功耗：四目标之一，但证据最弱

文章给出：一加 13（骁龙 8 至尊版）、CPU 4 线程、输入 1024/输出 128，采集电池电流电压、扣除闲置基线、3 轮，平均增量能耗约 **2.7 J/轮**；按 6000mAh 电池算，100 次约占 **0.3%**。

但技术报告中没有任何功耗/热评测章节（本报告已全文核对）。这意味着：

- 该数字只有单设备、单负载、3 轮，缺少方差、温度、频率、屏幕状态等条件；
- 没有 iPhone/iPad 功耗，没有持续调用下的温升曲线；
- 无法判断"100 次 ≤1%"目标在真实业务并发下是否成立。

[判断] 对一个把"功耗"列为四大目标之一的工作，这是最明显的证据缺口。见 §8。

5.6 与 llama.cpp 的对比：文章有，报告无

文章称：在骁龙 8 Gen 5、4bit 下，MNN 的 Prefill 是 llama.cpp 的 **1.51–2.90×**，Decode 是 **1.43–1.82×**。

本报告核对：**技术报告中没有 llama.cpp 对比章节或表格**，仅在 §5.2 提到"Following llama-bench"的测量方法。模型卡提供了 GGUF（F16/Q8_0/Q4_1）供 llama.cpp 使用，但未给对比数据。

[判断] 该结论**无法从报告或模型卡交叉验证**，只能视为团队未公开的额外测试。若用于选型，应自行在同设备、同量化、同 prompt 下复测。

---

6. 能力评测：数字背后的真实强弱项

6.1 评测设置

- 11 项：C-Eval、CMMLU、MMLU-Redux、BBH、GSM8K、MGSM-zh、MATH500、HumanEval、MBPP、IFEval、IFEval-zh；
- 后训练模型：知识用 zero-shot CoT，BBH 用 3-shot CoT，GSM8K/MATH500 用 4-shot CoT，MGSM-zh 用 8-shot CoT，代码与 IF 用 zero-shot；
- 基线：Gemma-3-270M、ERNIE-4.5-0.3B、LFM2.5-350M、SmolLM2-360M、Hunyuan-0.5B、MiniCPM4-0.5B、Qwen3-0.6B。

6.2 完整结果

Benchmark	MNN	INT4	Gemma-3-270M	ERNIE-4.5-0.3B	LFM2.5-350M	SmolLM2-360M	Hunyuan-0.5B	MiniCPM4-0.5B	Qwen3-0.6B
平均	**50.00**	**49.59**	6.42	31.99	31.28	18.43	31.22	46.81	44.28
C-Eval	38.86	40.27	6.84	32.17	32.10	23.77	35.29	**67.98**	44.06
CMMLU	40.11	39.65	4.94	33.91	31.65	24.87	35.69	**66.04**	44.50
MMLU-Redux	44.00	43.40	13.58	32.68	41.01	24.17	36.29	**52.85**	51.63
BBH	47.38	46.63	8.09	31.07	26.03	27.74	**54.65**	47.61	41.94
IFEval	**73.20**	74.12	20.15	44.92	72.09	32.16	24.58	52.13	56.93
IFEval-zh	**74.59**	72.49	17.02	32.87	44.06	25.64	25.87	30.30	50.35
GSM8K	**54.36**	52.84	0.00	31.24	33.66	11.60	36.01	42.46	45.49
MGSM-zh	**51.20**	48.80	0.00	25.20	18.00	3.60	34.80	37.60	40.00
MATH500	35.40	35.40	0.00	21.40	14.80	4.60	35.80	23.00	**45.20**
HumanEval	**53.66**	53.66	0.00	38.41	14.63	0.00	14.63	48.17	36.59
MBPP	37.20	38.20	0.00	28.00	16.00	24.60	9.80	**46.80**	30.40

6.3 怎么读这张表

结论 1：平均分领先不等于全面领先。

- MNN 平均 50.00，比最强 ~300M 基线 ERNIE（31.99）高 **18.01**，比更大的 MiniCPM4-0.5B（46.81）高 3.19、Qwen3-0.6B（44.28）高 5.72；
- 但**知识三项落后于知识型基线**：C-Eval 38.86 远低于 MiniCPM4 的 67.98（也低于 Qwen3 的 44.06）；CMMLU 40.11 远低于 66.04；MMLU-Redux 44.00 低于 MiniCPM4 的 52.85 与 Qwen3 的 51.63；
- 它的领先来自 **IF、中文数学、代码**，以及"没有明显塌方"的均衡性。

[判断] 这是典型的**"任务能力型"小模型**，不是"知识型"小模型。如果业务是知识问答、百科、客服 FAQ，0.3B 仍然不够；如果业务是意图识别、信息抽取、结构化输出、短代码/数学，它是越级的。

结论 2：专才 vs 通才。

- LFM2.5-350M 英语 IFEval 72.09 接近 MNN，但 MATH500 14.80、HumanEval 14.63；
- MiniCPM4-0.5B 知识极强（C-Eval 67.98），但 IFEval-zh 仅 30.30；
- Hunyuan-0.5B BBH 54.65 最高，但 IFEval 24.58；
- MNN 在 11 项里**全部进入前三**（报告称 top-3 on all 11、best on 5；从表内数据看，其他模型都至少有一项跌出前三。此处的"前三"不计自家 INT4 变体）。

[判断] 这种"无短板"比"单项冠军"更适合作为 App 的**统一底座**——因为 App 的业务任务是不可预测的长尾，单点极强、其他塌方的模型很难复用。

结论 3：基座模型只排第二。 Base 版平均 33.58，仅次于 ERNIE-4.5-0.3B Base 的 34.00（差 0.42），高于 LFM/SmolLM2/Gemma。但 Qwen3-0.6B Base（44.86）和 Hunyuan-0.5B-Pretrain（42.60）仍明显更强。[报告 §6.3]

[判断] 这说明 MNN 的**预训练底座并非全面领先**，它的最终优势很大程度上来自后训练（SFT Loop + MOPD + QAT）。这对复现者是重要提示：**买/用一个好底座不够，后训练配方才是差距所在**。

6.4 MOPD 的增益结构

任务	SFT 初始化	MOPD 后	增益
MGSM-zh	23.20	51.20	+28.00
IFEval-zh	49.42	74.59	+25.17
MATH500	13.00	35.40	+22.40
平均	35.79	50.00	+14.21

[判断] 增益集中在"教师被 RL 强化过的领域"，方向正确。但幅度之大也提示两种可能：一是 SFT 初始化在这些任务上确实欠拟合，蒸馏空间大；二是教师 RL 数据与评测集存在分布重叠。报告未给出教师训练数据与 benchmark 的去污染说明，这是需要追问的点。

6.5 统计显著性：小样本评测的噪声

- HumanEval 仅 **164 题**（公开数据集规模），单题约 **0.61 分**；一两题差异就是 1 分上下；
- MGSM-zh 约 250 题，单题约 0.4 分；
- IFEval 约 541 条，单条约 0.18 分；
- 报告未给置信区间，**无法判定 0.41 分是否统计显著**。但 11 项等权平均会把独立噪声压到约 0.61/√11 ≈ 0.18 分，0.41 约为 2 个标准误；再考虑各项高度相关、样本量差异大，更稳妥的结论是"差异很小、方向不稳定"，而不是"完全无损"。

[判断] 因此：

- "50.00 vs 49.59"不应解读为精确的"量化损失 0.41 分"，更准确的说法是"量化损失很小，且在评测噪声量级"；
- "HumanEval 53.66 vs Qwen3 36.59"差距足够大（28 题），是可信的；
- "C-Eval 38.86 vs Qwen3 44.06 / MiniCPM4 67.98"（C-Eval 题量大）是可信的知识短板；
- 但"MBPP 37.20 vs 38.20"这类 1 分差不要当真。

6.6 评测可信度评估

维度	评价
指标口径	清楚（shot 数、CoT、greedy）
基线复跑	**未明确说明**是否全部在同一 harness/prompt 下由作者统一复跑；小模型对 prompt 极敏感
去污染	**未说明**预训练/教师 RL 数据与 11 个 benchmark 的去重与泄漏检查
统计显著性	未报告置信区间或多次采样
平均分定义	11 项等权宏平均，未按业务重要性加权
可复现性	权重、GGUF、MNN 包齐全；评测脚本未随仓库提供（模型卡未列）

[判断] 数值大概率是真实跑出来的（有负结果、INT4 有升有降），但**多个基线的 0.00 更像 harness/prompt/输出解析失效，而非真实能力为零**。因此"平均分第一"的说服力应打一定折扣，尤其是基线是否同条件复跑、教师数据是否污染评测集这两点。

7. 两条应用验证链路

7.1 电商评论观点挖掘（外部榜单，可信度较高）

任务：给定中文美妆评论（品牌已用 `**` 匿名），抽取四元组 `(AspectTerms, OpinionTerms, Categories, Polarities)`。属性词/观点词必须原文复现，类别限定 13 类，极性三选一，未提及字段置空；一条评论可含多个观点。

为什么是个好探针：

- 四字段全对才计分，不能靠情感词表面匹配；
- 极性常由委婉/让步表达承载，惩罚表面线索模型；
- 输出是严格结构化列表，同时考指令遵循与格式稳定性；
- **由天池官方对隐藏测试集打分**，是外部、可复现的比较。

设置：

- 训练 3,229 条评论 / 6,633 个四元组（平均 2.05 个/条）；测试 2,237 条，标签隐藏；
- 所有 backbone 用**完全相同**的 LoRA SFT 配方（ms-swift、bf16、单张 AMD 卡、3 epoch、lr 1e-4、warmup 0.05、max_len 4096、batch 16、无梯度累积）；
- 关闭可选 reasoning 模式；Gemma-3 不支持 system role 时前置到首轮 user。

结果（F1，四字段精确匹配）：

Backbone	参数	F1
MNN-Pocket-0.3B	**0.3B**	**80.15**
Qwen3-1.7B	1.7B	78.59
Qwen3-0.6B	0.6B	77.47
Hunyuan-0.5B	0.5B	77.27
ERNIE-4.5-0.3B	0.3B	76.73
LFM2.5-350M	0.35B	76.19
MiniCPM4-0.5B	0.5B	75.27
LFM2.5-230M	0.23B	73.14
Gemma-3-270M	0.27B	72.23
SmolLM2-360M	0.36B	67.54

`[判断]` 这条链路是本报告认为**最有说服力的证据**：

1. 外部榜单、隐藏测试集，不能自说自话；
2. 所有模型同配方微调，差异归因于底座；
3. 0.3B 打赢 1.7B，说明在**有域内监督**的结构化任务上，容量不是瓶颈，**可适配性**才是；
4. 全场跨度 12.61 分，排序不随参数量单调——进一步说明"大"不等于"好用"。

这也解释了"8B 级效果"的真实含义：**在明确、可监督的业务任务上**，0.3B + LoRA 可以追平甚至超过数倍参数量的模型——本实验最大的对照是 1.7B，**并未验证 8B**；它是任务级结论，不是通用能力对标。

7.2 全端侧流式语音对话

系统由 5 个阶段 + 打断控制组成（全部用 MNN 本地运行）：

1. **音频前端**：AEC 回声消除、NS 降噪、AGC 自动增益（16kHz 单声道）；
2. **流式感知**：Silero VAD 语音活动检测 + X-ASR（INT8）流式转写；

3. **断句**：VAD 候选端点 + Smart Turn（语义完整性）+ 转写稳定性检查；
4. **生成**：INT4 MNN-Pocket-0.3B 流式生成，文本分段/归一化后送 TTS；
5. **语音输出**：MOSS-TTS-Nano 增量合成 + 播放队列；
6. **打断控制（非阶段）**：检测到新语音时，取消生成、取消合成、停止播放；播放信号回馈 AEC 作参考。

真机资源与时延：

指标	iPhone 13 Pro / A15	iPad Pro / M5	Android / 8 Gen 5
模型存储 (MB)	2,018	2,018	2,018
峰值内存 (MB)	1,591	1,858	1,704
ASR RTF	0.073	0.047	0.045
LLM 解码 (tok/s)	130.2	324.9	231.7
TTS RTF	0.653	0.366	0.651
理论 TTFT (ms)	~169	~46	~144
TTFT P50/P95 (ms)	568 / 585	472 / 481	523 / 535
理论 TTFA (ms)	~485	~171	~358
TTFA P50/P95 (ms)	**1,055 / 1,113**	**600 / 618**	**935 / 1,022**

> **理论值 vs 实测值**：理论值假设模型已预热、端点立即提交、从最早可用文本片段开始合成；实测值包含 ASR 尾部完整性、语义断句与 TTS 韵律稳定性的保护延迟，因此 P50 明显高于理论值。报告只给了假设、未给公式，理论值不可复算。
>
> 注意：**2GB 存储和 1.6–1.9GB 峰值内存是"整条语音链路"（ASR+TTS+VAD+LLM）**，不是语言模型本身。语言模型仍只是 173MB / 228MB。开发者做集成时必须按全链路预算，而不是按模型卡上的 173MB。

VoiceBench（9 项，统一 X-ASR 前端）：

Backbone	Overall
MNN-Pocket-0.3B	**45.44**
LFM2.5-350M	43.27
Qwen3-0.6B	41.15
MiniCPM4-0.5B	38.48
Gemma3-270M	36.18
ERNIE-4.5-0.3B	32.37
SmolLM2-360M	30.69
Hunyuan-0.5B	24.35

分项上，MNN 在 **BBH（58.90）、IFEval（64.11）、AdvBench（97.69）** 第一；但 **SD-QA 仅 5.75**（Qwen3 11.15、SmolLM2 15.78），是明显弱项。

[判断] 语音链路的工程完成度很高（流式、打断、AEC、语义断句都有），端到端 600–1,055ms 的首音频时延在端侧属于可用水平（无对照基线，具体阈值取决于业务）。但要注意：

- 评测的开放式任务由 **Qwen3.7-Max 打分**（3 票/样本），不是纯本地评测——部署离线，但评测用了云端 judge；
- SD-QA 的低分说明"语音输入 + 文档问答"仍是短板；
- 2GB 存储对轻量 App 是门槛。

8. 批判性评估：亮点、局限与待核实声明

8.1 真正的亮点

1. **约束驱动的协同设计**：架构、词表、量化、内核、草稿模型围绕同一组手机预算优化，而不是各自最优。
2. **词表裁剪的依赖感知实现**：保留特殊 token、byte-fallback、完整 BPE merge 链，工程细节到位。
3. **MOPD 的"训练多教师、部署单学生"**：在不增加端侧成本的前提下整合四域能力，是本文最有复用价值的方法。
4. **Self-OPD + QAT 的部署闭环**：INT4 与 BF16 的差距很小，分项有升有降、方向不稳定，说明量化做得很扎实。
5. **诚实的负结果**：Android OpenCL 聊天投机解码 0.88×、Android OpenCL 解码慢于 CPU、基座只排第二——这些都写进了报告，没有藏。
6. **外部榜单验证**：电商评论任务是第三方隐藏测试集，比自报 benchmark 可信。
7. **开放度**：Apache-2.0、Base/SFT/Instruct/INT4/GGUF/Dec、完整技术报告。
8. **端到端语音系统**：不是单点模型 demo，而是含打断/AEC/语义断句的完整链路，并给了 P50/P95。

8.2 局限与未解问题

#	问题	影响	证据
1	**知识能力是真短板**	不适合知识问答/百科/FAQ；C-Eval 38.86 vs MiniCPM4 67.98	`[报告 表7]`
2	**基线复跑条件未明确**	"平均分第一"可能受 prompt/harness 差异影响	`[报告 §6.1]`
3	**去污染未说明**	MOPD +22~28 的增益可能含评测集重叠；预训练/教师 RL 数据未做泄漏检查说明	`[报告 §4]`
4	**等权平均 + 小样本**	平均分不是能力指数；HumanEval 164 题，1 题 0.61 分	`[公开数据集规模/推算]`
5	**32K 上下文与 300MB 预算冲突**	BF16 KV 在 32K 约 448MiB；手机上 2K 稳妥、4K 接近上限	`[推算]`
6	**功耗无报告章节**	四目标之一只有文章单机数据，无法验证持续调用/温升	全文核对
7	**iPhone 18 Pro 350 tok/s 不可核验**	来自演示视频；报告基准表无该设备，无测试条件	`[文章/模型卡]`
8	**llama.cpp 对比不在报告**	1.51–2.90×/1.43–1.82× 无法交叉验证	全文核对
9	**"8B 级"无 8B 基线**	是设计目标/任务级说法，不是通用能力结论	`[文章]`
10	**安全评估单薄**	只有 VoiceBench 的 AdvBench；无红队/毒性/越狱评估	`[报告 §7.2]`
11	**数据来源与许可未讨论**	预训练/SFT 数据合规性无法评估	报告未提
12	**DFlash2 草稿模型开放情况不明**	文章称开源，但模型卡版本表未列该模型	`[模型卡]`
13	**仅中英**	多语言业务需另行评估	`[模型卡]`
14	**长上下文质量未评测**	只扩到 32K，没有 RULER/长文 QA 等质量指标	报告未提
15	**Android OpenCL 解码 138 < 200 目标**	目标只在部分后端成立；Android 应优先 CPU	`[报告 表4]`
16	**Android 单模型内存未测**	228MB 是 iPad 数，Android 预算需自行实测	`[报告 §5.2/§7.2]`
17	**工具调用未评测**	模板支持 tool_call/tool_response，但无 benchmark	`[chat_template]`

8.3 需要向团队追问的 9 个问题

1. 11 项 benchmark 的基线是否**全部由你们在同一 harness、同一 prompt、同一 shot 设置下复跑**？还是部分引用原论文/原模型卡？
2. 预训练语料、SFT 数据、四个教师的 RL 数据，是否对 11 个 benchmark 做过**去重/去污染**？MOPD 的巨大增益有多少来自"见过类似题"？
3. **32K 上下文在手机上的实测峰值内存**是多少？是否使用 KV 量化？在 300MB 预算下可用的最大上下文是多少？
4. iPhone 18 Pro 的 **350 tok/s** 的完整测试条件（上下文长度、后端、量化、是否含 prefill）是什么？为何未进报告基准表？
5. 与 **llama.cpp** 的对比是否有可复现的脚本/配置？为何未纳入技术报告？
6. 功耗测试的**完整方法**（采样频率、屏幕状态、温度、频率、方差）以及持续 10 分钟调用的温升曲线？
7. **DFlash2 草稿模型**是否已开放？在哪个仓库？与主模型的版本对应关系？
8. 是否有**长上下文质量**（RULER、LongBench 等）与**安全/越狱**评估结果？
9. Android OpenCL 解码 138 未达 200 目标，是继续优化还是官方推荐 Android 走 CPU？

---

9. 开发者上手指南

9.1 版本选择决策树

```
你要做什么?
├─ 手机/平板端通用对话、抽取、轻量 Agent
│    └─ MNN-Pocket-0.3B-MNN（INT4 W4A16 QAT, 173MB）★ 首选
├─ 需要自己量化/改结构/做研究
│    └─ MNN-Pocket-0.3B（BF16 safetensors, 723MB）+ llmexport
├─ 已在 llama.cpp 生态
│    └─ MNN-Pocket-0.3B-GGUF（F16 / Q8_0 / Q4_1）
├─ 分类 / 意图路由 / 候选选择（不需要生成）
│    └─ MNN-Pocket-0.3B-Dec（LoRA + 指针头，一次前向）
├─ 继续预训练 / 消融实验
│    └─ MNN-Pocket-0.3B-Base
├─ 研究 SFT 中间态
│    └─ MNN-Pocket-0.3B-SFT
└─ 需要更高生成吞吐
     └─ 主模型 + DFlash2 草稿（开放状态待确认）
```

9.2 三条部署路径

路径 A：MNN 端侧（推荐）

```
# 1）取模型（INT4 QAT 版）
#    ModelScope: MNN/MNN-Pocket-0.3B-MNN
#    包内: llm.mnn / llm.mnn.weight / llm_config.json / tokenizer.mtok / config.json

# 2）编译 MNN
```

```
git clone https://github.com/alibaba/MNN.git
cd MNN
mkdir build && cd build
cmake .. -DMNN_LOW_MEMORY=true -DMNN_BUILD_LLM=true -DMNN_SUPPORT_TRANSFORMER_FUSE=true
make -j

# 3) 跑 demo
./llm_demo /path/to/MNN-Pocket-0.3B-MNN/config.json prompt.txt
```

- 若要其他精度，从 BF16 safetensors 自行导出：

```
cd MNN/transformers/llm/export
python llmexport.py --path /path/to/MNN-Pocket-0.3B \
  --export mnn --quant_bit 4 --quant_block 64
```

- 移动端集成（Android/iOS）以 MNN 仓库示例工程与 MNN-LLM wiki 为准；编译宏与 `config.json` 参数见官方文档。

路径 B：Transformers（桌面/服务器验证）

```
from modelscope import snapshot_download
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = snapshot_download("MNN/MNN-Pocket-0.3B")
model = AutoModelForCausalLM.from_pretrained(model_id, device_map="auto", dtype="bfloat16")
tok = AutoTokenizer.from_pretrained(model_id)

messages = [{"role": "user", "content": "What is MNN?"}]
inputs = tok.apply_chat_template(
    messages, add_generation_prompt=True,
    return_tensors="pt", tokenize=True, return_dict=True
).to(model.device)

out = model.generate(**inputs, max_new_tokens=512,
                     do_sample=True, temperature=0.8, top_p=0.8, top_k=20)
print(tok.decode(out[0][inputs["input_ids"].shape[1]:], skip_special_tokens=True))
```

路径 C：llama.cpp

使用 `MNN-Pocket-0.3B-GGUF`：F16/Q8_0 来自 BF16 权重，Q4_1 来自 QAT 权重。`[模型卡]`

9.3 对话模板

ChatML 风格，**务必用** `tokenizer.apply_chat_template()`，不要手拼：

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
```

```
Hello!<|im_end|>
<|im_start|>assistant
```

模型卡中的 `[IM_START]` 是占位写法，真实控制符是 `<|im_start|>` / `<|im_end|>`（见 `chat_template.jinja`）。模板还支持工具调用：`<tools>...</tools>`、`<tool_call>{"name":...,"arguments":...}</tool_call>`、`<tool_response>...</tool_response>`。

9.4 微调建议

论文给出的可复用 LoRA 配方（来自电商评论任务）：

项	值
框架	ms-swift
精度	bf16
Epoch	3
LR	1e-4
Warmup ratio	0.05
Max length	4,096
Batch	16（单卡，无梯度累积）
LoRA	任务适配

`[判断]` 0.3B 模型单卡即可微调，实验成本极低，适合"多轮小步快跑"。但要注意：

1. **微调后自行导出 INT4 是 PTQ，不等于官方 QAT**，精度损失可能大于 0.41 分；对精度敏感的场景，建议保留 BF16 或自己做 QAT。
2. 数据格式要与 ChatML 模板一致；结构化任务建议在 system prompt 中**枚举候选标签**（论文的 13 类/3 极性写法可直接借鉴）。
3. 小模型对 prompt 极敏感，务必固定模板做 A/B，不要跨模板比较分数。
4. 关闭 reasoning 模式（如果底座支持）以匹配训练分布。

9.5 资源预算规划

场景	磁盘	峰值内存	时延参考
仅 LLM（512 ctx）	173MB	228MB	M5 466 tok/s；8 Gen5 CPU 251 tok/s
仅 LLM（4K ctx，估算）	173MB	~280MB	接近预算上限
仅 LLM（32K ctx，BF16 KV，估算）	173MB	~670MB	不建议在 300MB 预算内
投机解码草稿（DFlash2）	未公开	未公开	报告未给草稿体积/内存增量；共享 embedding/LM head 推测增量有限，但需实测
全端侧语音（ASR+TTS+LLM）	~2GB	1.6–1.9GB	TTFA 600–1,055ms

9.6 适合 / 不适合

适合：

- 意图识别、槽位抽取、评论/工单结构化、标签分类；

- 端侧轻量 Agent 的规划/工具调用（短链；模板原生支持 tool_call，但报告未评测工具调用准确率，需自行验证）；
- 中文短文本生成、改写、摘要；
- 语音助手的语言底座；
- 作为"统一小底座"替代多个任务专用小模型。

不适合：

- 知识问答、百科、法规/医疗等事实密集场景；
- 长程代码生成、复杂多步数学证明；
- 多语言（仅中英）；
- 需要 32K 长上下文且内存受限的手机场景；
- 高安全要求场景（缺红队/毒性评估）。

9.7 常见坑

1. **把 173MB 当成整条链路的存储**——语音链路是 ~2GB。
2. **默认 GPU 更快**——Android 上 OpenCL 解码可能慢于 CPU（138 vs 251）。
3. **默认开投机解码**——聊天负载在 Android OpenCL 上会负加速（0.88×）；CPU 路径未测。
4. **在手机上开 32K**——KV Cache 会吃掉数百 MB。
5. **跨 prompt 比较分数**——小模型对模板极敏感。
6. **微调后直接 PTQ 到 INT4 并期待 QAT 效果**——会有额外损失。

---

10. 行业定位与意义

10.1 0.3B 赛道的竞争维度

维度	通用小模型路线	MNN-Pocket 路线
优化目标	benchmark 平均分	应用资源预算 + 业务可用性
结构	通用 Transformer	硬件在环选型（层数/宽度/GQA/词表）
词表	继承大模型	依赖感知裁剪，换取深度
后训练	SFT + RLHF	SFT Loop + 多教师 OPD + QAT
部署	交给社区	自研引擎 + 内核 + 草稿模型 + 决策头
交付	一个 checkpoint	Base/SFT/Instruct/INT4/GGUF/Dec 全套
验证	自报 benchmark	外部榜单 + 端到端语音

[判断] 单看 0.3B checkpoint，复现门槛不高；真正的护城河是**"模型 + 引擎 + 量化 + 草稿 + 决策头 + 业务验证"的整条栈**。这也是为什么它更像"应用基础设施"而不是"论文模型"。

10.2 对端侧生态的三点影响

1. **"小模型 + 强后训练"继续挤压中等模型**：0.3B 在结构化任务上打赢 1.7B，说明很多业务不需要更大模型，只需要更好的域内适配。
2. **推理引擎成为模型竞争力的一部分**：同一份权重，MNN 与 llama.cpp 的差距（若文章数据成立）说明运行时优化仍有很大空间。

3. **"决策头/草稿模型"这类配套模型会成为标配**：一个负责"不生成、一次前向做选择"（Dec），一个负责"少生成、并行验证"（DFlash2），把 LLM 从"万能生成器"拆成"专用计算单元"。

10.3 与淘宝业务的关系

[判断] 这篇文章本质上是一次**业务需求反向定义模型**的展示：

- 淘宝/闲鱼的高频场景是评论理解、意图、搜索推荐、智能交互，不是开放域知识问答；
- 这些场景天然适合 0.3B + LoRA + 结构化输出；
- 端侧部署解决隐私、时延、云成本三个问题；
- Dec 变体直接对应"分类/候选选择"，说明团队是按业务形态设计模型族，而不是先训模型再找场景。

这套方法论对其他超级 App（内容、社交、本地生活）有直接借鉴意义。

---

11. 结论

MNN-Pocket-0.3B 是一次高质量、工程导向的端侧小模型实践。 它的价值不在"0.3B 拿了多少分"，而在于展示了一条可复制的路径：

> 用真机资源预算反推架构 → 用词表裁剪重分配参数 → 用五维数据桶做阶段配方 → 用 Agent 闭环补 SFT 短板 → 用多教师 OPD 整合能力 → 用 Self-OPD/QAT 无损落地 → 用引擎内核与草稿模型榨干硬件。

它适合谁：需要在手机/平板上做结构化理解、轻量生成、语音交互，且能接受"知识弱、仅中英、长上下文受限"的团队。

它不适合谁：做知识问答、复杂推理、多语言、长文档，或需要严格安全评估的场景。

最需要补的三块证据：

1. 数据去污染与基线复跑条件（决定"第一"的可信度）；
2. 32K 上下文的真实内存与质量（决定长上下文能否落地）；
3. 功耗/温升的持续负载评测（四目标之一，目前证据最弱）。

一句话建议：把它当作**"任务级大模型替代候选"**（本报告验证到 1.7B，未验证 8B），而不是"通用 0.3B 聊天模型"来用；先用电商评论这类有明确验收标准的任务做 POC，再决定是否扩大范围。

---

12. 附录

附录 A：文章说法 vs 报告/模型卡核对表

文章说法	报告/模型卡实际	核对结论
iPhone 18 Pro 离线 350 tok/s	报告基准表无该设备；模型卡仅有 demo 视频	⚠️ 无法从报告验证
iPad Pro M5 解码 466 tok/s	表4：466	✅ 一致
Prefill 8,721 tok/s	表4：8,721	✅ 一致
骁龙 8 Gen 5 CPU 解码 251 tok/s	表4：251	✅ 一致
部署体积 173MB	报告 173MB；仓库 166.1 MiB ≈ 174.2MB	✅ 约等于（差 0.7%，MiB/MB 换算）
（文章未提）BF16 权重 723MB	safetensors 头：存储 361.55M 参数（lm_head 与 embed 各一份）	ℹ️ 有效唯一参数仍为 302.8M
峰值内存 228MB	表3：228	✅ 一致
体积减少 49%	1−173/340 = 49.1%	✅ 正确
内存减少 47%	1−228/427 = 46.6%	✅ 正确
比 Qwen3-0.6B 快 1.63×	466/286 = 1.63	✅ 正确
词表 151K→64K（-57%）	65,536/151,936 = 43.1%	✅ 正确
4.1T token / 四阶段	3T+800B+200B+100B = 4.1T	✅ 一致
3.16M SFT 样本	表2 总计 3.16M	✅ 一致
MOPD 35.79→50.00	§6.4	✅ 一致
INT4 49.59（-0.41）	表7	✅ 一致
电商 F1 80.15	表10	✅ 一致
语音 TTFA 600/935/1,055ms	表11	✅ 一致
2.7 J/轮、100 次 0.3%	**报告无功耗章节**	⚠️ 仅文章，无法核验
比 llama.cpp 快 1.51–2.90×/1.43–1.82×	**报告无此对比**	⚠️ 无法核验
16K star / 200+ 模型 / 百万下载	**报告无此数据**	⚠️ 仅文章（背景数据，非技术主张）
DFlash2 草稿模型已开源	模型卡版本表未列	⚠️ 待确认
达到 8B 级效果	报告无 8B 基线	⚠️ 设计目标，非实测

附录 B：复现清单

- □ 下载 `MNN/MNN-Pocket-0.3B-MNN`（INT4）与 `MNN/MNN-Pocket-0.3B`（BF16）
- □ 编译 MNN（`MNN_LOW_MEMORY` / `MNN_BUILD_LLM` / `MNN_SUPPORT_TRANSFORMER_FUSE`）
- □ 在目标机型跑 512 token prefill/decode，记录 5 次中位数
- □ 实测峰值内存（含 KV），分别在 512 / 2K / 4K / 8K 上下文
- □ 用 `apply_chat_template` 跑 11 项中的 2–3 项，核对报告数值
- □ 用相同 LoRA 配方在自有业务数据上做 POC
- □ 若需长上下文，先测 KV 量化与内存水位
- □ 若需更高吞吐，按负载开关投机解码（聊天负载先测再开）

附录 C：关键术语

术语	含义
TPP	Tokens Per Parameter，每参数训练 token 数
GQA	Grouped Query Attention，分组查询注意力
MOPD	Multi-Teacher On-Policy Distillation，多教师 On-Policy 蒸馏
OPD	On-Policy Distillation，学生自己生成、教师在其前缀上监督
Reverse KL	反向 KL，mode-seeking，强调教师主峰
QAT	Quantization-Aware Training，量化感知训练
W4A16	4-bit 权重、16-bit 激活
AL	Accepted Length，投机解码每轮平均接受 token 数
TTFT / TTFA	首 token 时延 / 首音频时延
RTF	Real-Time Factor，实时率（<1 表示快于实时）

附录 D：参考链接

- GitHub：https://github.com/alibaba/MNN
- Hugging Face：https://huggingface.co/taobao-mnn/MNN-Pocket-0.3B
- ModelScope：https://modelscope.cn/models/MNN/MNN-Pocket-0.3B
- 技术报告：https://modelscope.cn/models/MNN/MNN-Pocket-0.3B/resolve/master/MNN-Pocket-0.3B-Technical-Report.pdf
- 模型合集：https://modelscope.cn/collections/MNN/MNN-Pocket-03B
- 电商评论榜单：https://tianchi.aliyun.com/competition/entrance/532421/information

---

本报告基于公开资料整理，`[推算]` 与 `[判断]` 部分为独立分析，不代表 MNN 团队观点。建议在选型前按附录 B 在目标机型复测。